

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**СИСТЕМА ЗА АНАЛИЗ НА ТРАНСПОРТНАТА
СИГУРНОСТ И ЗА УПРАВЛЕНИЕ НА
ДОПУСКАНЕТО НА ВРЪЗКИ ПРИ УСЛОВИЯ
НА ЛАВИНЕН ТРАФИК**

КУРСОВ ПРОЕКТ

по дисциплина „Мрежова и информационна сигурност“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Румен Трифонов

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Стандарти и политики	3
1.2. Заплахи и тяхната класификация	3
1.3. Протоколни спецификации	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	6
2.1. Изисквания към сглобяването и тяхната тежест	6
2.2. Източник на данните за анализа	6
2.3. Обхват на собствената реализация	7
2.4. Стратегия за допускане на връзки	8
2.5. Език и инструментариум	8
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	10
3.1. Слой за четене и разчитане	10
3.2. Слой за проверка на сертификатната верига	12
3.3. Слой за допускане на връзки	13
3.4. Модел на заплахата и граници на системата	15
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	16
4.1. Изисквания към сборката и последиците от тях	16
4.2. Обща структура на изходния текст	16
4.3. Четене на байтове	17
4.4. Разчитане на ръкостискането	18
4.5. Разчитане на DER и на сертификата	18
4.6. Изграждане и проверка на пътя	19
4.7. Механизми за допускане	19
4.8. Стенд за натоварване	20
4.9. Тестове	20
4.10. Генериран стенд вместо записан трафик	21

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	22
5.1. Среда за провеждане	22
5.2. Набори от данни	23
5.3. Измервани величини и как се получават	24
5.4. Изисквания за валидност	26
5.5. Очаквани резултати	27
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	29
6.1. Разчитане на ръкостискането и проверка на веригата	29
6.2. Цена на едно решение за допускане	30
6.3. Поведение при лавинен трафик	32
6.4. Отчитане на предварителните очакванията	34
ЗАКЛЮЧЕНИЕ	36
ЛИТЕРАТУРА	38
ПРИЛОЖЕНИЯ	40

УВОД

Сигурността на един мрежов обмен днес се крепи на два механизма, които потребителят не вижда и обикновено не проверява. Първият е протоколът за защита на транспортния слой, който договаря шифрите и ключовия материал. Вторият е инфраструктурата на публичните ключове, която свързва един публичен ключ с едно име и позволява тази връзка да бъде отменена. И двата механизма са описани в публични спецификации, и двата се реализират от малък брой библиотеки, и точно затова техните свойства рядко се проверяват самостоятелно. Приема се, че щом връзката е установена, тя е защитена, а щом библиотеката не е върнала грешка, веригата от сертификати е валидна.

Настоящият курсов проект изхожда от обратната постановка. Договорените параметри на една връзка са наблюдаема величина и могат да бъдат прочетени от самия обмен. Валидността на една сертификатна верига е резултат от алгоритъм, който е записан в [1] и може да бъде реализиран самостоятелно, вместо да бъде приет наготово. Устойчивостта на един сървър срещу лавинен трафик е свойство, което се измерва, а не се декларира. Проектът разработва система, наречена Sentinel, която прави и трите неща: чете и разчита ръкостискането по TLS 1.3, проверява сертификатната верига по описания в стандарта алгоритъм за изграждане на път, и управлява допускането на нови връзки при условия на претоварване. Към трите се добавя четвърто изискване, което се оказва не по-малко определящо за облика на решението: системата да се сглобява на чужда машина без предварителна подготовка, тоест без нито една задължителна външна зависимост.

Целта на проекта е да се изгради работеща и измерима система за анализ на транспортната сигурност и за управление на допускането на връзки, чиито решения са проследими до конкретни изисквания на действащите стандарти, а цената на всеки защитен механизъм е измерена, а не предположена.

За постигането на целта са формулирани следните **задачи**:

1. преглед на действащите политики, стандарти и класификации на заплахите в мрежовата и информационната сигурност, с акцент върху публикациите на ISO, ITU, ETSI, ENISA и NIST;
2. анализ на възможните проектни решения по всяка от трите подзадачи и обосновка на направения избор;

3. проектиране на слоева архитектура, в която разчитането на протокола, проверката на сертификатите и управлението на допускането са отделени един от друг;
4. програмна реализация на C++20 без задължителни външни зависимости, при която логиката на анализа и на проверката е собствена, а проверките, които не могат да бъдат проведени без криптографска основа, се отчитат изрично като непроведени;
5. изграждане на методика за измерване на цената, която всеки механизъм за допускане внася в закъснението и в пропускателната способност;
6. експериментална проверка в собствена изолирана среда и анализ на получените резултати.

Обхватът на проекта е защитен. Системата анализира, проверява и допуска. Тя не съдържа средства за проникване, за заобикаляне на защити или за атака срещу чужда инфраструктура. Всички опити с лавинен трафик се провеждат единствено срещу собствения стенд на проекта, изграден върху локален интерфейс и контейнери, и това ограничение е част от методиката, а не пожелание към нея.

Структура на изложението. Раздел I разглежда предметната област, стандартите и класификацията на заплахите. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката на експериментите, а Раздел VI получените резултати и техния анализ. Заключението обобщава предимствата и недостатъците на решението и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Мрежовата и информационната сигурност е област, в която нормативните документи изпреварват реализациите. Преди да се проектира каквато и да е система, е нужно да се установи срещу какво се защитава тя и кой документ определя дали защитата е достатъчна. Този раздел прави този преглед в три стъпки: първо стандартите и политиките, после класификацията на заплахите, накрая конкретните протоколни спецификации, върху които стъпва реализацията.

1.1. Стандарти и политики

Рамката за управление на информационната сигурност се задава от серията ISO/IEC 27000. За настоящата задача пряко приложима е ISO/IEC 27033-1 [2], която въвежда понятията за мрежова сигурност, разделя мрежата на зони и описва как се избират защитни мерки за границата между тях. Архитектурният речник, върху който стъпват тези документи, идва от препоръките на ITU-T. Форматът на сертификата за публичен ключ, върху който се крепи цялата проверка в Раздел III, е дефиниран в препоръката ITU-T X.509 [3], а профилът му за приложение в интернет е зададен от IETF в [1].

От страна на техническите указания NIST издава документи, които се четат като изисквания към конфигурация, а не като теория. За избора и настройката на реализациите на TLS това е [4], а за защитните стени и политиката им [5]. Двата документа са особено полезни за настоящия проект, защото формулират проверими твърдения. Изискване от вида „да не се договаря версия под TLS 1.2“ е нещо, което анализаторът на ръкостискането може да провери автоматично.

Публикациите на ETSI, които попадат в обхвата на дисциплината, засягат преди всичко доставчиците на удостоверителни услуги и изискванията към тях. Конкретните документи, които влизат в този преглед, се определят и цитират в **[TODO: конкретни публикации на ETSI, сверени по номер и версия]**.

1.2. Заплахи и тяхната класификация

Годишният доклад на ENISA [6] подрежда наблюдаваните заплахи по категории и проследява как относителната им тежест се променя. За настоящия проект значение

имат три от тях. Първата е отказът на услуга, включително разпределеният му вариант, при който източниците на трафик са множество компрометирани машини. Втората е злоупотребата с инфраструктурата на публичните ключове, при която проверяващата страна приема сертификат, който не би трябвало да приеме. Третата е манипулацията на разрешаването на имена, при която жертвата получава правилен по форма отговор, но с подменено съдържание.

Защитата на уеб приложенията стои встрани от трите, но е част от учебното съдържание и е неотделима от транспортния слой. Класификацията на OWASP [7] подрежда рисковете при уеб приложенията и е използвана тук като контролен списък при определянето какво остава извън обхвата на проекта. Проектът не се занимава с логиката на приложението, а със слоя под него.

За защитата на разрешаването на имена изходната точка е [8], който въвежда изискванията към DNSSEC и обяснява защо липсата на удостоверяване на произхода на отговора е структурен, а не реализационен проблем. Отношението между тази защита и настоящата система е косвено, но съществено: ако името е разрешено до грешен адрес, проверката на сертификатната верига остава единственият механизъм, който може да открие подмяната.

1.3. Протоколни спецификации

Основната спецификация за анализатора е [9]. TLS 1.3 се различава от предходните версии по това, че съкращава ръкостискането и шифрира по-голямата част от него, което пряко определя докъде може да стигне пасивният анализ. Наблюдаемата част от обмена и границата, отвъд която анализаторът вижда само шифриран текст, се уточняват в Раздел III.

Механизмите за защита срещу лавина от заявки за установяване на връзка са описани в [10]. Документът разглежда систематично защо лавината от SYN сегменти е евтина за източника и скъпа за получателя, и излага контрамерките, включително подхода с криптографски изчислени начални последователни номера. Този документ е основата на Раздел V, защото описва не само мерките, но и техните странични ефекти, а именно те се измерват в настоящия проект.

Накрая, [11] описва как QUIC въвежда ключовата обмяна на TLS 1.3 в собствен транспортен слой. Спецификацията е включена в прегледа, защото показва, че

разглежданите тук механизми не са свързани с конкретен транспорт, и защото поставя въпроса дали анализаторът може да бъде разширен и към този случай.

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Всяка от трите подзадачи допуска повече от едно решение и изборът между тях предопределя както обхвата, така и това, което изобщо може да бъде измерено. Разделът излага разгледаните възможности и мотивите за приетото решение. Където реализацията впоследствие наложи различен избор от първоначално предвидения, това е записано тук заедно с причината.

2.1. Изисквания към сглобяването и тяхната тежест

Преди трите подзадачи стои едно решение, което ги предопределя и трите: от какво има право да зависи сборката. Разгледани са два подхода.

Първият е обичайният: необходимите библиотеки се обявяват и се доставят от управител на пакети или се изтеглят при настройването на сборката. Реализацията е по-кратка, защото готовите библиотеки поемат работа, но сборката изисква подготвена машина и мрежа.

Вторият е сборката да зависи единствено от компилатор за C++20 и CMake. Тогава всяка функция, която иначе би дошла отвън, трябва или да бъде написана, или да отпадне заедно с това, което зависи от нея.

Приема се вторият подход. Основанието е, че хранилището е публично и трябва да може да бъде сглобено от непознат без предварителна подготовка. Сборка, която изисква управител на пакети, е сборка, която никой не пуска. Приетото решение има цена и тя е платена на три места, изброени в Раздел 4.1: отпада библиотеката за прихващане на пакети, отпада външната рамка за тестове и отпада проверката на подписа. Първите две отпадания не отнемат нищо от задачата. Третото отнема и затова е разгледано отделно в Раздел 2.3.

2.2. Източник на данните за анализа

Първата възможност е анализът да се извършва вътре в приложението, чрез обратни извиквания на криптографската библиотека, която вече е разчела съобщенията. Реализацията тогава е кратка, но резултатът описва само собствените връзки на приложението и е обвързан с версията на библиотеката.

Втората възможност е пасивно наблюдение на трафика чрез библиотека за прихващане на пакети, върху жив интерфейс или върху записан файл. Анализът обхваща чужди на приложението връзки, а записаният файл прави измерването възпроизводимо. Цената е една задължителна външна зависимост, което влиза в противоречие с Раздел 2.1.

Третата възможност, която е и приетата, е слой за разчитане да приема просто изглед към байтове, тоест `std::span<const std::uint8_t>`. Откъде идват байтовете не е негова работа: записан файл, сокет или генериран от тестовите набор изглеждат еднакво. Тази възможност запазва двете предимства на втората, защото анализът пак не е обвързан с приложението и входът пак е фиксиран, а премахва зависимостта. Загубеното е удобството да се чете формат на записан трафик направо; това е отделна задача за разчитане на формат, а не свойство на анализа, и е записано като посока за по-нататъшна работа.

2.3. Обхват на собствената реализация

Криптографските примитиви са зона, в която собствената реализация е грешка. Изборът на алгоритми, устойчивостта им спрямо атаки по странични канали и правилността на самите изчисления са резултат от години рецензии. Първоначалният замисъл затова предвиждаше примитивите да се вземат от утвърдена библиотека.

При реализацията това изискване влезе в противоречие с Раздел 2.1, който е по-силен. Разгледани са три изхода.

Първият е да се приеме задължителна криптографска зависимост, което отменя изискването за сглобяване без подготовка. Вторият е да се напише собствена аритметика с големи числа, което е точно грешката, срещу която предупреждава първият абзац. Третият е проверката на подписа да не се извършва и това да бъде обявено при всяка верига, редом с проверките, които са проведени.

Приема се третият изход. Той е по-малкото зло по следната причина: първите два или отнемат свойството, заради което хранилището е използваемо, или заменят рецензирана реализация с нерецензирана. Третият не отнема нищо и не заменя нищо, а само стеснява обхвата и го обявява. Съществено е, че обявяването е част от изхода, а не добавка към него. Мълчаливото пропускане на проверка прави отчета по-опасен от липсата на отчет, защото читателят би приел, че веригата е проверена криптографски.

Логиката над примитивите остава предмет на проекта, както беше предвидено. Разчитането на съобщенията на ръкостискането, изграждането на пътя до доверен корен, проверката на ограниченията върху имената и на разширенията, както и обработката на състоянието на отмяната, се реализират самостоятелно по описанието в [1].

Проверката на отмяната е сведена до локално поддържан списък от серийни номера. Изтегляне на списък с отменени сертификати или запитване по [12] изискват изходяща мрежова връзка, каквато системата няма и по устройство не може да има. Политиката при липсваща информация за отмяна остава явен параметър, както е разгледано в Раздел 3.2.2.

2.4. Стратегия за допускане на връзки

Разгледани са четири механизма за защита при лавинен трафик, всеки от които има различна цена. Криптографски изчислените стойности без съхранение на състояние, описани в [10], премахват нуждата от състояние за незавършена връзка, но изискват допълнителен обмен. Ограничаването на честотата по източник е евтино, но е безполезно срещу разпределен източник и вредно, когато честният и лавинният трафик делят един адрес. Допускането срещу изчислителна задача прехвърля цената върху клиента, но внася закъснение при всяко първо свързване. Отчитането на таблицата с връзките е по-скоро измерителен, отколкото защитен механизъм, но без него останалите три не могат да бъдат оценени.

Приема се всичките четири да бъдат реализирани и включвани поотделно, вместо да се избере един. Причината е, че въпросът на проекта не е кой механизъм е най-добър, а колко струва всеки от тях. Отговорът изисква сравнение при еднакви условия, тоест изисква всички да съществуват в една и съща система.

Към четирите се добавя политика на плавно влошаване, която не е самостоятелен механизъм, а правило, което вдига цената на останалите три с нарастването на запълването на таблицата. Тя е добавена, защото без нея механизмите работят с една и съща настройка при празен и при пълен сървър, а точно при пълен сървър защитата има значение.

2.5. Език и инструментариум

Изборът на C++20 е продиктуван от три съображения. Разчитането на протокола работи с байтови буфери, за което `std::span` дава невладеещ изглед без копиране и без загуба на дължината. Измерването на закъснението изисква предвидимо време на

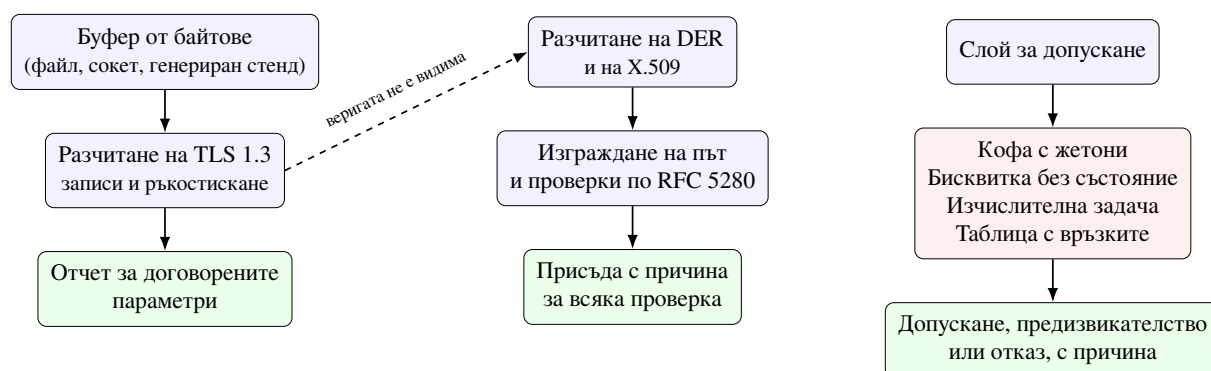
изпълнение, без паузи от събиране на боклука. И накрая, стандартната библиотека покрива нужното, така че изискването от Раздел 2.1 е изпълнимо без да се пише всичко наново.

Алтернативите бяха разгледани и отхвърлени: Rust заради по-малкия личен опит на автора в рамките на срока на проекта, а езиците със среда за изпълнение и събиране на боклука заради влиянието им върху точно тази величина, която проектът измерва.

За сглобяването е избран CMake, защото е стандартът за този инструментариум и защото позволява отделните цели за компилация да направят зависимостите между слоевете видими в конфигурацията, а не само в текста. Тестовите се регистрират в CTest поотделно, като списъкът им се произвежда от самия изпълним файл след свързването, за да не може да се размине с кода.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е разделена на слоеве, така че всеки от тях да може да бъде проверен самостоятелно. Слойът за четене на байтове доставя данни, независимо от техния произход. Слойът за разчитане превръща байтовете в структури на протокола. Слойът за проверка взема решение за валидността на сертификатната верига. Слойът за допускане решава дали една нова връзка да бъде обслужена. Общата схема на слоевете и потокът от данни между тях се дават на (Фиг. 1).



Фигура 1. Слоевете на системата и потокът на данните между тях. Прекъснатата стрелка означава, че пасивният наблюдател не получава сертификатната верига от самото ръкостискане по TLS 1.3 и тя трябва да дойде от друг източник.

3.1. Слой за четене и разчитане

Входът на слоя за разчитане е `std::span<const std::uint8_t>`, тоест невладеещ изглед към чужд буфер. Откъде идват байтовете не е работа на този слой: записан файл, сокет или генериран от тестовите набор изглеждат еднакво. Това е съзнателно проектно решение и то има две последици. Първата е, че проектът не се нуждае от библиотека за прихващане на пакети, за да бъде полезен и проверим. Втората е, че всяко измерване е възпроизводимо, защото входът е фиксиран поредица от байтове, а не поведението на мрежата.

Четенето на всяко поле минава през `byte_reader`, който проверява дължината преди всяко четене. При недостиг на байтове четецът се маркира като отказал и всяко следващо четене е празна операция; причината за първия отказ се запазва, защото по-късните каскадни откази казват по-малко за входа от този, който ги е предизвикал. Нищо в този слой не хвърля изключения: повреденото или нарочно изкривено съобщение

е нормален вход за такъв код, а не изключителна ситуация, и обработката му трябва да е еднакво евтина при всеки пакет.

Съществено проектно решение е границата на видимостта. В TLS 1.3 само първата част от ръкостискането е в явен вид, а останалата е шифрована с ключове, които пасивният наблюдател няма. Затова анализаторът докладва това, което наистина се наблюдава, брои неразчетеното и изброява изрично какво не е видял. Наблюдаемите и скритите параметри се дават в (Табл. 1).

Таблица 1. Наблюдаеми и скрити параметри на ръкостискането по TLS 1.3

Параметър	Наблюдаем	Източник или причина
Версия на протокола	да	разширение <code>supported_versions</code> в <code>ServerHello</code>
Шифрова съвкупност	да	поле <code>cipher_suite</code> в <code>ServerHello</code>
Избрана група за ключова обмяна	да	разширение <code>key_share</code> в <code>ServerHello</code>
Заявено име на сървъра	да	разширение <code>server_name</code> в <code>ClientHello</code>
Договорен протокол по ALPN	да	разширение <code>ALPN</code> в <code>ServerHello</code>
Предложени от клиента версии, групи, схеми за подпис	да	разширения в <code>ClientHello</code>
<code>EncryptedExtensions</code> , <code>Certificate</code> , <code>CertificateVerify</code> , <code>Finished</code>	не	защитени с ключовете на ръкостискането
Сертификатна верига на сървъра	не	носи се в защитеното съобщение <code>Certificate</code>
Сертификат на клиента	не	същото
Билети за възобновяване и ранни данни	не	същото

Редът в (Табл. 1) за версията заслужава отделно внимание. По [9] полето `legacy_version` в `ServerHello` остава със стойност за TLS 1.2 при всяка връзка по TLS 1.3, за да преминава през междинни устройства. Анализатор, който чете това поле, съобщава TLS 1.2 за всяка връзка по TLS 1.3. Затова системата чете версията от разширението и записва в отчета откъде я е взела.

3.2. Слой за проверка на сертификатната верига

Проверката се извършва на два етапа. Първият изгражда пътя от сертификата на края до доверен корен чрез обхождане в дълбочина по множеството от кандидат-издатели. Разглеждат се всички възможни продължения, а не само първото намерено, защото първият намерен път до доверен корен често не е този, който минава проверките. Вторият етап прилага върху всеки пълен път проверките, изисквани от [1]. Списъкът им се дава в (Табл. 2).

Таблица 2. Проверки върху сертификатния път и източникът на изискването

Проверка	Източник	Какво отхвърля
Свързване по име	RFC 5280, 6.1	прекъсване между издател и субект
Срок на валидност	RFC 5280, 4.1.2.5	изтекъл или още невязъл в сила
Основни ограничения	RFC 5280, 4.2.1.9	издател, който не е удостоверител
Ограничение върху дължината	RFC 5280, 4.2.1.9	път с повече междинни звена от позволеното
Предназначение на ключа	RFC 5280, 4.2.1.3	удостоверител без keyCertSign
Разширено предназначение	RFC 5280, 4.2.1.12	ключ, негоден за сървърно удостоверяване
Съответствие на името	RFC 6125, 6.4	име, непокрито от subjectAltName
Ограничения върху имената	RFC 5280, 4.2.1.10	име извън позволеното поддърво
Критични разширения	RFC 5280, 6.1	неразпознато критично разширение
Отмяна	RFC 5280, 5	сериен номер в списъка на отменените
Подпис на издателя	RFC 5280, 6.1	не се извършва, вж. по-долу

3.2.1. Проверка на подписа и защо тя се отчита като непроведена

Първоначалният замисъл предвиждаше криптографските примитиви да се вземат от утвърдена библиотека, както е записано в Раздел 2.3. При реализацията това изискване влезе в противоречие с друго, по-силно: сборката на проекта не трябва да зависи от нищо освен компилатор за C++20 и CMake, за да може да бъде повторена на чужда

машина без подготовка. Проверката на подписа изисква аритметика с големи числа, чието самостоятелно реализиране би било точно грешката, срещу която предупреждава Раздел 2.3.

Приетото решение е проверката да не се извършва и това да бъде записано в отчета, редом с проверките, които са проведени. Всеки отчет съдържа ред със състояние SKIP и причината. Обратното решение, тоест мълчаливо пропускане, би направило отчета по-опасен от липсата на отчет: читателят би приел, че веригата е проверена криптографски. Отчет, който скрива непроведена проверка, е по-лош от липсващ отчет.

3.2.2. Политика при липсваща информация за отмяна

Отмяната се проверява срещу локално поддържан списък от серийни номера. Изтегляне на CRL или запитване по [12] не се извършва, защото и двете изискват изходяща мрежова връзка, а системата няма такава. Разгледаните варианти при липсваща или остаряла информация за отмяна и последиците от тях се дават в (Табл. 3).

Таблица 3. Варианти на политиката при недостъпен източник за отмяна

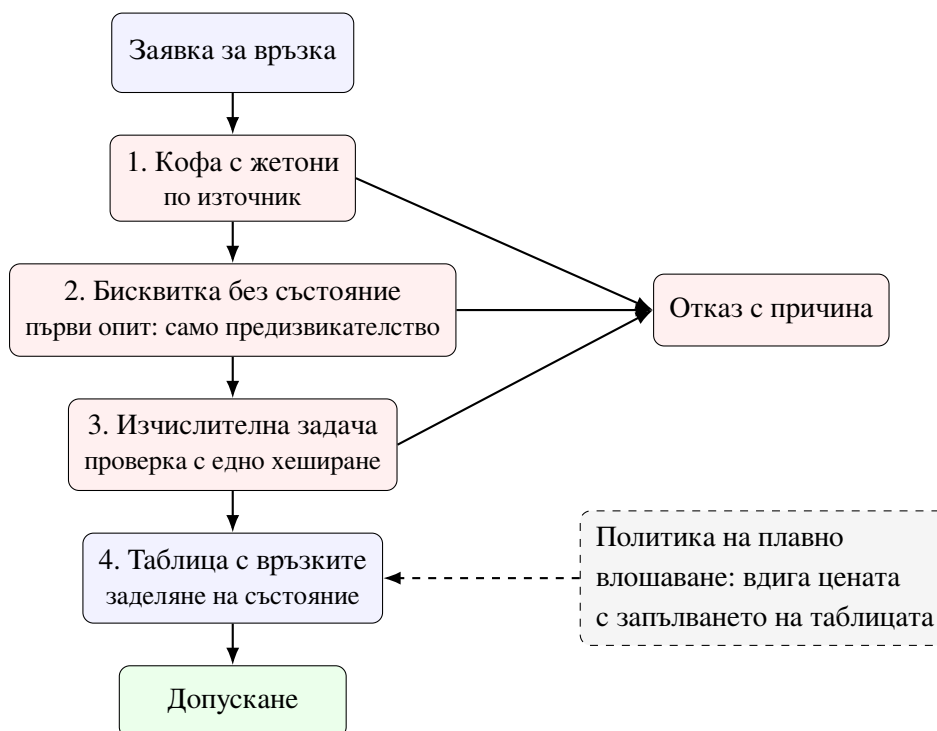
Вариант	Поведение	Последица
Отказ навън	веригата се приема, проверката се отчита като непроведена	отменен сертификат минава, докато източникът е недостъпен
Отказ навътре	веригата се отхвърля	недостъпен източник спира и напълно редовния трафик, което е отказ на услуга

И двата варианта са защитими и всеки от тях е грешен в някакво разгръщане. Точно затова изборът е параметър на проверката, а не подразбиране, скрито в кода. Каквото и да е избрано, отчетът съдържа ред за отмяната с изричното състояние: PASS при проверен списък, FAIL при намерен серийен номер или при отказ навътре, SKIP при отказ навън.

3.3. Слой за допускане на връзки

Слоят за допускане стои пред сървъра и решава дали да отдели ресурс за нова връзка. Проектиран е като верига от независими механизми, всеки от които може да бъде включен или изключен, без това да засегне останалите. Такава организация е необходима,

защото измерването в Раздел V изисква всеки механизъм да бъде оценен и поотделно, и в съчетание с другите. Редът на механизмите в веригата се дава на (Фиг. 2).



Фигура 2. Ред на механизмите в слоя за допускане. Състояние се заделя едва на стъпка 4, след като по-евтините проверки са минали.

Редът не е произволен. Кофата с жетони е най-евтината проверка и стои първа. Бисквитката без състояние следва, защото тя е механизмът, който позволява целият флот от незавършени опити да не струва нищо: при първи опит сървърът не заделя нищо, а връща стойност, която клиентът трябва да върне обратно и с това да докаже, че получава на адреса, който твърди, че има. Едва след като бисквитката е потвърдена, се стига до заделяне на състояние.

Отчитането на таблицата с връзките е общо за цялата верига. То поддържа броя на незавършените и на установените връзки, разпределението им по източници и времето, прекарано във всяко състояние. Тези величини са едновременно вход за решенията и изход за измерването. Когато таблицата се напълни, изхвърля се най-старата незавършена връзка: незавършената връзка е по-евтиното нещо за загубване и при лавина тя почти сигурно е част от лавината.

Политиката на плавно влошаване чете запълването на таблицата и разделя работата на три режима: нормален под 50 на сто, повишен до 85 на сто и критичен над 85 на сто. При повишен режим един източник харчи два жетона вместо един и трудността на

изчислителната задача се вдига с два бита. При критичен режим се допускат само клиенти, които вече са върнали бисквитка, а трудността се вдига с четири бита.

3.4. Модел на заплахата и граници на системата

Пълното изложение на модела се дава в (Табл. 4). Системата приема, че наблюдателят няма достъп до частните ключове и не може да дешифрира защитената част от обмена. Приема също, че опитите с лавинен трафик се провеждат в среда, която е изцяло под контрола на автора: приемащата страна слуша единствено на локалния интерфейс, а генераторът на натоварване е част от същия процес.

Таблица 4. Модел на заплахата: актьор, способност и мярка на системата

Актьор	Способност	Мярка
Пасивен наблюдател	чете трафика, няма ключове	извън обхвата: системата е на мястото на наблюдателя, не се защитава от него
Издател на неправомерен сертификат	подписва сертификат за чуждо име	проверка на името, ограничения върху имената, основни ограничения
Страна, представяща негодна верига	подрежда произволни сертификати	изграждане на път до доверен корен, проверка на всяко звено
Източник на лавина от един адрес	евтино открива връзки	кофа с жетони по източник, отчитане на таблицата
Разпределен източник на лавина	евтино открива връзки от много адреси	бисквитка без състояние, изчислителна задача, плавно влошаване
Нападател срещу самата машина	изпълнение на код на машината	извън модела на заплахата
Нападател срещу приложния слой	злоупотреба над транспорта	извън обхвата, вж. [7]

Отделно ограничение, което не е заплахата, а свойство на реализацията: проверката на подписа не се извършва, както е обяснено в Раздел 3.2.1. Следователно системата не открива верига, чиито структурни полета са правилни, но подписът е подправен. Това ограничение е записано в отчета на всяка проверка и се повтаря в Заключение.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Реализацията е на C++20 и се изгражда с CMake. Проектът е разделен на библиотека, която съдържа цялата логика, и на два изпълними файла, които са тънки обвивки над нея. Разделянето е направено заради тестовете: всичко, което заслужава проверка, живее в библиотеката и се вика пряко от тестовете, без да се минава през команден ред.

4.1. Изисквания към сборката и последиците от тях

Приетото изискване е сборката да не зависи от нищо освен от компилатор за C++20 и CMake. Без управител на пакети, без изтегляне на код при настройване, без задължителна външна библиотека. Изискването не е въпрос на вкус: хранилището е публично, а сборка, която изисква предварителна подготовка на машината, е сборка, която никой не пуска.

Изискването има три конкретни последици в кода, всяка от които е записана тук, защото всяка от тях промени първоначалния замисъл:

1. **Няма `librcp`.** Слой за разчитане приема `std::span` от байтове. Откъде идват байтовете не е негова работа. Това не отнема нищо от анализа и премахва цяла зависимост, вж. Раздел 2.2.
2. **Няма `GoogleTest`.** Изпълнителят на тестовете е в хранилището, около сто реда в `tests/check.hpp` и `tests/test_main.cpp`, вж. Раздел 4.9.
3. **Няма криптографска библиотека и следователно няма проверка на подписа.** Тя се отчита като непроведена при всяка верига, вж. Раздел 3.2.1.

Единствените външни библиотеки, които се свързват, са тези, които операционната система доставя сама: `ws2_32` под Windows, нищо допълнително под обичайните Unix-подобни системи, и нишките през `find_package(Threads)`.

4.2. Обща структура на изходния текст

Публичните заглавни файлове стоят в `include/sentinel/`, реализацията в `src/`, тестовете в `tests/`, а двете програми в `apps/`. Модулите и обемът им се дават в (Табл. 5).

Таблица 5. Модули на проекта, обем и зависимости между тях

Модул	Редове	Съдържание	Зависи от
<code>bytes.hpp</code>	124	четец с проверка на границите, тип за резултат	—
<code>siphash</code>	117	SipHash-2-4	<code>bytes</code>
<code>tls</code>	768	записи, ръкостискане, разширения, отчет	<code>bytes</code>
<code>der</code>	424	строг разчитащ на DER	<code>bytes</code>
<code>x509</code>	589	структура на сертификата	<code>der</code>
<code>chain</code>	495	изграждане на път и проверки	<code>x509</code>
<code>admission</code>	621	четирите механизма и политиката	<code>siphash</code>
<code>net</code>	258	сокети, скрити зад един интерфейс	—
<code>harness</code>	332	сървър и клиент за натоварването	<code>net</code> , <code>admission</code>
<code>fixtures</code>	766	генериран стенд: байтове и сертификати	<code>tls</code> , <code>siphash</code>
<code>experiment</code>	595	измерванията	всички горни
Тестове	1449	82 случая в седем набора	всички
Програми	261	<code>sentinel_demo</code> , <code>sentinel_bench</code>	<code>experiment</code>

4.3. Четене на байтове

Целият вход минава през `sentinel::byte_reader`. Всяко четене проверява дължината предварително; при недостиг четецът се маркира като отказал и всяко следващо четене е празна операция. Причината за първия отказ се запазва и не се презаписва от следващите, защото каскадната грешка казва по-малко за входа от тази, която я е предизвикала. Листинг 1 в Приложение 6.4 показва механизма.

Върнатият тип е `result<T>`, който съдържа или стойност, или причина за отказа. `std::expected` е от C++23, а проектът е на C++20, така че този тип е малкият му заместник. Изключения по пътя на разчитането няма.

4.4. Разчитане на ръкостискането

`tls::scan_stream` обхожда записите, брои ги по вид и слепва съдържанието на всички записи от вид `handshake` в един буфер, преди да го нарязва на съобщения. Редът е съществен: едно съобщение може да се разпростре върху няколко записа, а няколко съобщения могат да делят един запис. Разчитач, който приравнява един запис на едно съобщение, разчита правилно обичайния случай и сгрешава интересния. Тестът `tls_reassembles_a_handshake_message_split_across_two_records` подава едно и също ръкостискане в два вида, цял и разкъсан, и изисква договорените параметри да съвпадат.

Дължината на един запис се проверява срещу границата от [9], раздел 5.1, преди да бъде копиран какъвто и да е байт. Без тази проверка изпращачът решава колко памет заделя получателят.

Отчетът съдържа три части: договорените параметри, наблюденията по политика и списъка на ненаблюдаемостта. Наблюденията по политика са проверими твърдения, взети от [4]: договорена версия под TLS 1.2, шифрова съвкупност извън петте на TLS 1.3, предложени остарели съвкупности, наличие на сигнала за понижаване по [9], раздел 4.1.3.

4.5. Разчитане на DER и на сертификата

Разчитачът на DER отхвърля три неща, които не могат да се появят в съответстващ сертификат: неопределена дължина, неминимално кодирана дължина и формата с висок номер на етикета. Отхвърлянето не е педантизъм. Разчитач, който приема няколко кодирания на една и съща стойност, е начинът, по който две реализации стигат до различно мнение какво пише в един сертификат. По същата причина се отхвърлят неминимално кодирани цели числа и стойност на `BOOLEAN`, различна от `0x00` и `0xff`.

Времената се разчитат по [1], раздел 4.1.2.5. Двущифрената година в `UTCTime` се тълкува по правилото на стандарта: 50 и нагоре означава двадесети век. Обръщането на това правило прави изтекъл сертификат валиден за още седемдесет и пет години, затова то има отделен тест.

Разчитачът на сертификата пропуска правилно двете полета с уникален идентификатор от версия 2, което е необходимо, за да се стигне до блока с разширенията на правилното място. Неразпознато критично разширение не се игнорира, а се записва в списък, който проверката на пътя после отхвърля. Това е изискването на [1], раздел 6.1.

4.6. Изграждане и проверка на пътя

Изграждането на пътя е обхождане в дълбочина по множеството от кандидат-издатели, с ограничение върху дължината и с отчитане на вече посещенията, за да не се получи безкраен цикъл при кръстосано подписани сертификати. Доверените корени се пробват преди представените сертификати, защото най-краткият път до корен е обичайният случай и опитът с него пръв прави търсенето евтино.

Всяко отхвърляне на кандидат се записва с причина, така че при неуспех системата обяснява защо нито един път не е приет, вместо просто да върне отказ. Резултатът от проверката е списък от редове с три възможни състояния: PASS, FAIL и SKIP. Третото състояние е същественото нововъведение спрямо обичайния интерфейс от вида „валиден или не“: то позволява на системата да каже коя проверка не е била проведена и защо, вместо да я преброи като успешна.

Съответствието на името следва правилата на RFC 6125, раздел 6.4.3, и всяко от тях има отделно твърдение в тестовите, защото всяко от тях е предмет на действително разминаване между реализации: най-много един заместващ знак, само в най-лявата съставка, никога през точка, и поне две съставки след него, за да не може *.com да покрие каквото и да е.

4.7. Механизми за допускане

Четири механизма са реализирани зад общ интерфейс с едно решение и една причина. Веригата се конфигурира при стартиране и не се променя по време на измерване, за да не се смесват режими в един набор от данни.

Бисквитката без състояние е `siphash24` върху източника, стойността на клиента и груб времеви прозорец, съкратена до 32 бита, тоест до ширината, която едно поле за пореден номер предлага. Сървърът не пази нищо между двата обмена: той пресмята бисквитката отново, когато клиентът я върне. Търпи се изоставане с един прозорец, иначе всеки клиент, чийто отговор пресича границата на прозореца, се отказва без причина.

Изчислителната задача изисква `siphash24` върху избрано от клиента число да има зададен брой водещи нули. Проверката от страна на сървъра е едно хеширане, а търсенето от страна на клиента е средно 2^k хеширания при k бита трудност. Точно това несиметрично съотношение е целта на механизма, и то е измерено в Раздел VI.

Таблицата с връзките разделя незавършените от установените. При запълване се изхвърля най-старата незавършена връзка. Това е проектното решение на таблицата: незавършената връзка е по-евтиното нещо за загубване, а при лавина тя почти сигурно е част от лавината. Ако всички записи са установени, нова връзка се отказва, вместо да се изхвърли работеща.

Отчитането не използва заключване и не е разделено по нишки. Първоначалният замисъл предвиждаше разделена по нишки структура; при реализацията се оказа излишен: приемащият цикъл е една нишка, а общ брояч зад мютекс би поставил самото измерване вътре в измерваното. Ограничението е записано в заглавния файл: един приемащ цикъл владее един контролер.

4.8. Стенд за натоварване

Разликата между платформите е затворена в двойката `net.hpp` и `net.cpp`: Winsock под Windows, сокети на Бъркли навсякъде другаде. Нищо над този заглавен файл не знае на коя платформа работи и никакъв системен заглавен файл не влиза в останалата част от проекта.

Слушателят се свързва единствено с адреса на локалния интерфейс. Това не е подразбиране, а границата на опита: генераторът на натоварване в това хранилище не може да достигне нищо освен този процес.

Сървърът затваря пръв. Така клиентът е пасивната страна на затварянето и не задържа временния си порт след това, което пази дълъг опит от изчерпване на диапазона от портове. Записът в таблицата за отчитане обаче се задържа за определено време след затварянето на сокета, за да се моделира състоянието, което истински сървър пази след допускане; без това таблицата никога не би се напълнила и политиката на плавно влошаване не би се задействала.

4.9. Тестове

Изпълнителят на тестовете е в хранилището. Регистрацията е чрез статичен обект, а един макрос `TEST(име)` свързва тялото с името. Неуспешно твърдение се записва и изпълнението продължава, така че една счупена очаквана стойност не скрива следващите двадесет.

Свързването с CTest е направено така, че списъкът от случаи да не може да се разминава с кода: след свързването изпълнимият файл сам изписва `add_test` за всеки свой случай в генериран файл, който CTest прочита. Резултатът е 82 отделни записа в CTest, а не един общ. Разпределението им по набори се дава в (Табл. 6).

Таблица 6. Набори от тестове и какво проверява всеки от тях

Набор	Случаи	Предмет
<code>test_bytes</code>	7	граници при четене, запазване на първата причина, SipHash срещу публикуваните вектори
<code>test_tls</code>	12	договорени параметри, слепване през записи, сигнал за понижаване, съкратени и неправилни съобщения
<code>test_der</code>	11	неопределена и неминимална дължина, висок номер на етикет, обектни идентификатори, правилото за века
<code>test_x509</code>	9	полета на сертификата, заместващи знаци, поддървета на имената, критични разширения
<code>test_chain</code>	16	всички генерирани случаи, всяка проверка поотделно, политиката при отмяна
<code>test_admission</code>	20	кофа с жетони, бисквитка, изчислителна задача, изхвърляне от таблицата, влошаване
<code>test_net</code>	7	сокети от край до край, липса на състояние при неотговарящ клиент, ограничаване по източник

4.10. Генериран стенд вместо записан трафик

Целият входен материал се генерира от самото хранилище, поле по поле срещу [9] и [1]. Нищо не е прихванато от истинска връзка и никакъв истински сертификат не е вграден никъде. Това има три предимства: материалът е възпроизводим от изходния текст, може да му бъде въведен по един нарочен дефект, и не съдържа чужди данни.

Сертификатите носят синтетичен публичен ключ и запълващ низ на мястото на подписа. Ключова двойка не съществува и подпис не се изчислява. Това е съгласувано с проверката, която отчита подписа като непроверен: нищо в системата не чете тези байтове. Списъкът на генерираните случаи и очакваният изход за всеки от тях се дава в (Табл. 8).

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

Разделът описва как се получават резултатите от Раздел VI. Той е написан преди измерванията, а не след тях, за да може погрешно очакване да остане видимо и да бъде отчетено като погрешно.

5.1. Среда за провеждане

Всички опити се провеждат в изолирана среда, изградена изцяло от автора. Клиентът и сървърът работят в един процес върху локалния интерфейс. Слушателят се свързва единствено с адреса на локалния интерфейс, така че генераторът на натоварване не може да достигне нищо освен този процес. Никакъв трафик не напуска машината и никаква външна услуга не участва. Това ограничение е задължително и не подлежи на облекчаване. Конфигурацията на машината се дава в (Табл. 7).

Таблица 7. Конфигурация на средата, върху която са получени резултатите

Компонент	Стойност
Процесор	AMD Ryzen 5 3600, 6 ядра, 12 логически процесора, до 3950 MHz
Оперативна памет	15,9 GiB
Операционна система	Windows 11 Pro N, версия 10.0.26200
Компилятор	g++ 15.2.0, MinGW-w64, целева тройка x86_64-posix-seh
Система за сборка	CMake 4.3.2 с генератор Ninja 1.13.2
Режим на сборка	Release, -O3 -DNDEBUG, предупреждения -Wall -Wextra -Wpedantic
Стандарт на езика	C++20, без разширения на компилатора
Външни зависимости	няма, освен ws2_32 от операционната система
Часовник	std::chrono::steady_clock, 10 ⁹ отброявания в секунда

Машината е обща, а не отделена за измерване. Няма закрепване към ядро, няма заключване на честотата и по време на опитите на нея работят и други процеси. Това е ограничение на средата и то е отчетено в правилата за валидност в Раздел 5.4, вместо да бъде премълчано.

5.2. Набори от данни

Целият входен материал се генерира от самото хранилище, поле по поле срещу [9] и [1]. Нищо не е прихванато от истинска връзка и никакъв истински сертификат не е вграден. Изборът има три основания: материалът е възпроизводим от изходния текст, може да му бъде въведен по един нарочен дефект, и не съдържа чужди данни.

Анализаторът на ръкостискането се проверява върху три потока: пълен обмен по TLS 1.3 със ServerHello, разкъсано между два записа; сървър, отговорил с TLS 1.2 и поставил сигнала за понижаване, докато клиентът е предложил TLS 1.3; и съобщение, прекъснато вътре в записа, който го носи.

Проверката на веригата се провежда върху собствена йерархия от сертификати с по един нарочен дефект на случай. Случаите и очакваният изход за всеки от тях се дават в (Табл. 8). Очакваният изход е записан заедно със случая при неговото построяване, тоест преди да бъде пуснат.

Таблица 8. Случаи за проверка на сертификатна верига и очакваният изход за всеки

Случай	Очакван изход	Основание
Изрядна верига	приема се	всички изпълними проверки минават
Заместващ знак покрива една съставка	приема се	*.svc.example.test покрива api.svc.example.test
Заместващ знак през точка	отхвърля се	заместващият знак не се разпростира през точка
Изтекъл сертификат на края	отхвърля се	срокът е затворен преди отправното време
Име, непокрито от сертификата	отхвърля се	нито един dNSName не покрива искания хост
Издател, който не е удостоверяващ	отхвърля се	basicConstraints не казва cA
Път по-дълъг от позволеното	отхвърля се	pathLenConstraint на издателя е надхвърлено
Ключ само за подписване на код	отхвърля се	extKeyUsage изключва сървърно удостоверяване
Сериен номер в списъка на отменените	отхвърля се	намерен в списъка за отмяна
Неразпознато критично разширение	отхвърля се	RFC 5280, раздел 6.1
Име в позволеното поддърво	приема се	ограниченията върху имената са спазени
Име извън позволеното поддърво	отхвърля се	издателят няма право да удостоверява това име
Няма път до доверен корен	отхвърля се	нито представен, нито доверен сертификат го издава

5.3. Измервани величини и как се получават

Измерванията са три групи, всяка с отделна цел.

Пропускателна способност на разчитането и на проверката. Затворен цикъл в един процес, без сокети. Измерват се наносекунди за една операция при разчитане на пълен обмен по TLS 1.3, при разчитане на един сертификат и при изграждане и проверка на верига от три сертификата. Преди всяко измерване се изпълнява загряващ период, чиито резултати се изхвърлят: първото преминаване плаща за страничните грешки на паметта,

за обучението на предсказателя на преходи и за установяването на разпределителя на паметта, а измерването му би отчело началото на процеса, а не кода.

Цена на едно решение за допускане. Затворен цикъл в един процес. Времевата стойност, подавана на контролера, се увеличава с една микросекунда на решение. Замразен часовник би изпразнил кофата с жетони след нейната вместимост и би превърнал остатъка от цикъла в измерване на пътя на отказа, което не е търсената величина. Всяко решение, което не е допускане, се брои отделно; ако броят е различен от нула, редът смесва два пътя през кода и това се отпечатва вместо да бъде осреднено мълчаливо.

Изчислителната задача се измерва от двете страни поотделно: търсенето от страна на клиента и проверката от страна на сървъра. Двете величини се плащат от различни страни и смесването им в едно число би скрило точно несиметричността, заради която механизмът съществува.

Поведение при лавинен трафик. Действителни TCP връзки върху локалния интерфейс. Едновременно работят две нишки с честни клиенти, които довършват обмена, и четири нишки с лавинен трафик, които отварят връзка, получават предизвикателството и никога не отговарят. Снемат се броят на допуснатите връзки за секунда, закъснението при установяване като медиана и деветдесет и пети персентил, общият брой решения за секунда, върховото запълване на таблицата с връзките и заделената памет за състояние.

Измерва се и базова конфигурация, при която нито един механизъм не е включен. Без нея числата за останалите конфигурации нямат отправна точка. Параметрите на опита се дават в (Табл. 9).

Таблица 9. Параметри на експеримента

Параметър	Стойност
Повторения на конфигурация	11
Продължителност на едно повторение под натоварване	600 ms
Времеви бюджет на едно повторение при разчитане	250 ms
Времеви бюджет на едно повторение при решение	150 ms
Загряване преди разчитането на ръкостискане	512 преминавания
Загряване преди разчитането на сертификат	2048 преминавания
Загряване преди проверката на верига	512 преминавания
Нишки с честни клиенти	2
Нишки с лавинен трафик	4
Вместимост на таблицата с връзките	2048 записа
Задържане на запис след допускане	20 ms
Трудност на изчислителната задача	12 бита
Вместимост на кофата с жетони	4096
Скорост на попълване на кофата	2 000 000 в секунда
Продължителност на времевия прозорец на бисквитката	2 s

Скоростта на попълване на кофата е нарочно поставена над това, което стендът може да предложи от един адрес. Измерваната величина е цената на ограничителя, когато той допуска, а не цената на отказа. Върху локалния интерфейс всички клиенти делят един адрес на източника, така че ограничение, което действително захапва, би задушило честните клиенти заедно с лавината. Ограничител, който захапва, се проверява отделно в тестовите.

5.4. Изисквания за валидност

Измерването се приема за валидно само ако са изпълнени следните условия.

Първо, генераторът на натоварване не трябва да бъде тясното място. Проверява се по броя на решенията за секунда: ако той пада заедно с броя на допуснатите връзки,

ограничението е у приемника; ако остава постоянен, докато допуснатите намаляват, ограничението е в механизма.

Второ, разсейването между повторения трябва да остане под предварително определен праг. Праговете се дават в (Табл. 10). Като мярка се използва междуквartilният размах, отнесен към медианата, а не пълният размах. Причината е, че пълният размах върху единадесет повторения се определя от едно-единствено отклонение, а на обща машина едно отклонение означава планировчика на операционната система, а не кода. Пълният размах се отпечатва винаги, редом с междуквartilния, защото скриването на най-лошото повторение би било нечестният начин една таблица да изглежда спокойна.

Таблица 10. Прагове за приемане на набор от повторения

Величина	Праг за междуквartilния размах
Наносекунди за операция при разчитане и проверка	25 %
Наносекунди за едно решение за допускане	35 %
Допуснати връзки за секунда под натоварване	40 %

Ред, който надхвърли своя праг, се отпечатва с надпис, че не бива да бъде докладван, и се разглежда като такъв в Раздел VI. Той не се осреднява в число, което изглежда спокойно.

Трето, всички повторения се провеждат в един и същ процес и в един и същ ред, а конфигурацията не се променя по време на едно повторение, за да не се смесват режими в един набор от данни.

5.5. Очаквани резултати

Очакванията се записват тук преди измерването и се отчитат изрично в Раздел 6.4, включително когато не се потвърждават.

1. **O1.** Механизмът без съхранение на състояние ще намали значително заделената памет и върховото запълване на таблицата при лавина, за сметка на допълнителен обмен.
2. **O2.** Ограничаването по източник ще има пренебрежимо закъснение и няма да раздели честния трафик от лавинния, когато двата делят един адрес на източника.

3. **O3.** Изчислителната задача ще внесе най-голямото закъснение при първо свързване и ще бъде силно несиметрична: много хеширания у клиента срещу едно у сървъра.
4. **O4.** Собствената цена на решението за допускане ще бъде малка спрямо цената на една ТСП връзка, така че наблюдаваното намаление на пропускателната способност ще идва от формата на протокола, а не от изчислението.
5. **O5.** Разчитането на едно ръкостискане ще бъде от порядъка на единици микросекунди, тоест разчитането няма да е тясното място при никое разумно натоварване.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя получените резултати в реда на трите подзадачи и ги тълкува. Всяко число идва от изпълнение на програмата `sentinel_bench` върху машината от (Табл. 7), с командата

```
sentinel_bench --duration-ms 600 --repeat 11
```

Пълният изход на това изпълнение, включително всички единадесет повторения на всеки ред, е записан в хранилището като `docs/measurements/bench_run.txt` и е приложен в Приложение 6.4. Всяка обобщена стойност по-долу може да бъде проследена до стойностите, от които е получена.

6.1. Разчитане на ръкостискането и проверка на веригата

Резултатите от разчитането се дават в (Табл. 11). Докладваната стойност е медианата от единадесет повторения, а разсейването е дадено и като междуквартилен размах, и като пълен размах, по правилото от Раздел 5.4.

Таблица 11. Пропускателна способност на разчитането и на проверката

Операция	ns за операция	операции/s	МКР	размах
Разчитане на обмен по TLS 1.3, 2448 В	7 765,4	128 776	11,5 %	31,9 %
Разчитане на сертификат, 984 В	8 575,7	116 608	8,0 %	15,8 %
Изграждане и проверка на верига от 3	17 330,4	57 702	6,7 %	13,7 %

И трите реда са под своя праг от 25 на сто и се докладват. В други единици първият ред е 300,6 MiB/s върху обмена от 2448 байта, а вторият 109,4 MiB/s върху сертификата от 984 байта.

Проверката на верига от три сертификата струва 17,3 микросекунди, тоест два пъти повече от разчитането на един сертификат. Разликата не е в разчитането, защото сертификатите се разчитат веднъж и се подават разчетени: тя е в изграждането на пътя и в единадесетте проверки върху него, като част от тях изграждат канонично представяне на отличителните имена за сравнение. Числото е важно със своя порядък: при 57 хиляди

проверки на верига в секунда на едно ядро проверката на сертификати не е тясното място при ниско натоварване, което една машина може да поеме по TCP.

Присъдите по случаите от (Табл. 8) съвпадат с очакваните за всичките тринадесет случая. Изходът на `sentinel_demo` завършва с реда „13 of 13 cases produced the verdict they were built for“. Съответствието се проверява и автоматично от теста `chain_every_generated_case_produces_the_verdict_it_was_built_for`, който е част от набора и се изпълнява при всяка сборка. Случай, при който системата приема верига, която трябва да отхвърли, няма; ако имаше, той щеше да бъде разгледан отделно, защото е дефект в сигурността, а не разминаване в числата.

(Табл. 12) показва състоянието на единадесетте проверки за изрядната верига. Двата реда със състояние `SKIP` са съществено в тази таблица: те са проверките, които не са проведени, и системата ги обявява вместо да ги преброи като успешни.

Таблица 12. Състояние на всяка проверка при изрядна верига от три сертификата

Проверка	Състояние	Забележка
Свързване по име	PASS	три сертификата без прекъсване
Срок на валидност	PASS	всички покриват отправното време
Основни ограничения	PASS	всеки издател е удостоверявател
Ограничение върху дължината	PASS	не е надхвърлено
Предназначение на ключа	PASS	краят: подпис и шифроване на ключ
Разширено предназначение	PASS	<code>serverAuth, clientAuth</code>
Съответствие на името	PASS	съвпада с <code>dNSName</code>
Ограничения върху имената	SKIP	никой издател по пътя не ограничава имена
Критични разширения	PASS	всички са разпознати
Отмяна	PASS	няма сериен номер в списъка
Подпис на издателя	SKIP	няма криптографска основа; двата подписа не са проверени

6.2. Цена на едно решение за допускане

(Табл. 13) дава цената на едно решение вътре в процеса, без сокет по пътя. Това е собствената цена на механизма, отделена от цената на връзката.

Таблица 13. Цена на едно решение за допускане, измерена вътре в процеса

Конфигурация	ns за решение	МКР	размах	у клиента
Без механизъм (база)	8,7	40,7 %	50,7 %	—
Отчитане	93,9	8,8 %	15,1 %	—
Отчитане + ограничаване	94,9	9,4 %	18,7 %	—
Отчитане + бисквитка	124,9	13,7 %	31,9 %	—
Отчитане + бисквитка + задача	147,8	7,3 %	49,0 %	0,05 ms
Всички включени	136,0	32,2 %	54,2 %	0,06 ms

Първият ред надхвърля своя праг от 35 на сто и по правилото от Раздел 5.4 **не бива да бъде докладван като точна стойност**. Причината е ясна и не е в кода: при базовата конфигурация решението се състои от увеличаване на брояч и връщане, тоест единици наносекунди, а върху обща машина разсейването на планировчика е от същия порядък. Числото се запазва тук с този надпис, вместо да бъде премахнато или загладено. Порядъкът му, единици наносекунди, е достатъчен за извода, който се прави от него.

Останалите пет реда са в границите си. От тях се четат три величини:

- **Отчитането струва около 85 ns**, разликата между 8,7 и 93,9. Това е вписването и заличаването в хеш-таблицата с връзките, тоест цената на заделеното състояние.
- **Ограничаването по източник струва около 1 ns над отчитането**, тоест 94,9 срещу 93,9. Разликата е под разсейването на двата реда и от нея не бива да се вади заключение освен това, че цената е малка. Числото важи, когато ограничителят допуска.
- **Бисквитката струва около 31 ns**, тоест 124,9 срещу 93,9. Това са две извиквания на SipHash-2-4, а при отказ на проверката и още едно за търпимия предходен прозорец.
- **Проверката на изчислителната задача струва около 23 ns**, тоест 147,8 срещу 124,9, което е едно хеширане.

Несиметричността на изчислителната задача е измерена пряко. При трудност от 12 бита търсенето у клиента отнема средно **4279 хеширания** и **0,05 ms** на едно предизвикателство, докато проверката у сървъра е едно хеширане, тоест около 23 ns. Отношението между двете е малко над 2000. Ако трудността се вдигне с един бит, очакваният брой хеширания у клиента се удвоява, а цената у сървъра остава същата: точно това е свойството, заради което механизмът съществува, и то е причината политиката на плавно влошаване да вдига трудността, а не да въвежда нова проверка.

Редът „Всички включени“ дава 136,0 ns, тоест по-малко от реда без ограничаване по източник. Разликата е в рамките на разсейването на двата реда, чиито междуквартилни размахи са 32,2 и 7,3 на сто. От нея не се вади заключение. Отбелязва се, защото премълчаването на неудобно подреден ред е точно начинът, по който една таблица става недостоверна.

6.3. Поведение при лавинен трафик

(Табл. 14) е основният резултат на проекта: цената на всеки механизъм под действително натоварване, измерена спрямо базовата конфигурация. Всички редове са в границите си по разсейване.

Таблица 14. Цена на механизмите за допускане при лавинен трафик спрямо базата

Конфигурация	допуснати/s	спрямо базата	p50, us	p95, us	върхова таблица	памет, В
Без механизъм (база)	4305	100,0 %	454,2	575,9	0	0
Отчитане	3614	83,9 %	524,9	762,8	287	7552
Отчитане + ограничаване	3450	80,1 %	516,3	858,3	255	7912
Отчитане + бисквитка	1919	44,6 %	987,8	1390,2	48	1504
Отчитане + бисквитка + задача	1671	38,8 %	1094,2	1840,1	44	1408
Всички включени	1791	41,6 %	1027,1	1692,8	43	1384

6.3.1. Собствената цена на механизма е незначителна

Съпоставката между (Табл. 13) и (Табл. 14) дава най-съществения извод на измерването. Собствената цена на едно решение е от 9 до 148 наносекунди. Цената на една допусната връзка под натоварване е от 454 до 1094 *микросекунди*. Разликата е около четири порядъка. Изчислението, което механизмът извършва, е невидимо на фона на работата по установяването и затварянето на една TCP връзка.

Следователно намалението на пропускателната способност в (Табл. 14) не идва от изчислението. То идва от формата на протокола, което се вижда пряко в колоната с решенията за секунда в необобщения изход: базата взема 12 919 решения в секунда, а конфигурацията с бисквитка взема 11 532, тоест почти същото. Приемникът е еднакво зает и в двата случая. Това, което се променя, е колко от тези решения завършват с допусната връзка.

6.3.2. Бисквитката без състояние

Бисквитката намалява допуснатите връзки до 44,6 на сто от базата и удвоява закъснението. Причината е пряка: за да бъде допусната, една връзка изисква два обмена вместо един, тъй като първият опит получава само предизвикателство и се затваря. Цената е една допълнителна TCP връзка на допускане, а не изчисление.

Срещу тази цена се получава следното. Върховото запълване на таблицата пада от 287 записа на 48, тоест шест пъти, а заделената памет за състояние от 7552 на 1504 байта, тоест пет пъти, при едно и също количество лавинен трафик. Четирите нишки с лавина в конфигурацията с бисквитка не оставят нищо след себе си: всяка от техните връзки получава предизвикателство, не отговаря и не води до заделяне. Същото свойство се проверява и пряко от теста `harness_a_client_that_never_answers_the_cookie_leaves_no_state_behind`, който подава шестдесет неотговарящи и десет отговарящи клиента и изисква върхът на таблицата да не надхвърли десет.

Това е сделката, която механизмът предлага, изразена в измерени числа: половината пропускателна способност при нормални условия срещу шесткратно намаление на състоянието, което лавината може да наложи. Дали сделката си струва, зависи от това дали ограничението е пропускателната способност или паметта, и това е решение за конкретното разгръщане, не за проекта.

6.3.3. Ограничаване по източник

Ограничаването по източник дава 80,1 на сто от базата срещу 83,9 на сто за самото отчитане. Разликата от 3,8 процентни пункта е в рамките на разсейването на двата реда, чиито междуквартилни размахи са 11,6 и 13,2 на сто, и от нея не се вади заключение.

По-съществено наблюдение е качествено, а не числено. Върху локалния интерфейс честните клиенти и лавинните клиенти делят един адрес на източника, така че ограничител, който захапва, би задушил и двата вида еднакво. Това не е недостатък на стенда, а точното проявление на известното ограничение на механизма: кофата с жетони по източник разделя трафика само дотолкова, доколкото източниците са различни. Срещу разпределен източник тя не работи, а срещу споделен адрес тя вреди. Че механизмът все пак действа, когато източниците са различни, се проверява отделно от теста `harness_rate_limiting_refuses_a_single_source_that_goes_too_fast`.

6.3.4. Изчислителна задача и плавно влошаване

Добавянето на изчислителната задача върху бисквитката сваля допуснатите връзки от 44,6 на 38,8 на сто и вдига медианата на закъснението от 987,8 на 1094,2 микросекунди. Разликата от около 106 микросекунди се сравнява добре с измереното време за търсене у клиента от 0,05 ms, тоест 50 микросекунди, като остатъкът се пада на по-бавното обслужване при по-натоварен приемник.

Конфигурацията с всички механизми дава 41,6 на сто, тоест малко над конфигурацията само с бисквитка и задача. Обяснението е в политиката на плавно влошаване: при това натоварване запълването на таблицата остава ниско, режимът остава нормален и политиката не вдига трудността. Тя не помага и не пречи при тези условия. Политиката се задейства при по-високо запълване, което е проверено отделно от теста `admission_degradation_raises_the_difficulty_as_the_table_fills`, но не се проявява в тази конфигурация на опита. Това е ограничение на настоящото измерване и е записано като такова, а не приписано на механизма.

6.4. Отчитане на предварителните очакванията

Всяко очакване от Раздел 5.5 се отчита изрично в (Табл. 15).

Таблица 15. Отчитане на предварителните очаквания спрямо измерените стойности

Очак.	Исход	Измерване, което води до оценката
O1	потвърдено	върхова таблица от 287 на 48 записа, състояние от 7552 на 1504 байта, срещу допълнителен обмен и падане до 44,6 % от базата
O2	потвърдено	80,1 % срещу 83,9 % за самото отчитане, разлика в рамките на разсейването; върху споделен адрес механизмът не разделя честния от лавинния трафик
O3	потвърдено	4279 хеширания и 0,05 ms у клиента срещу едно хеширане и около 23 ns у сървъра, отношение над 2000
O4	потвърдено	9 до 148 ns за решение срещу 454 до 1094 us за връзка, около четири порядъка разлика; решенията за секунда остават 12 919 срещу 11 532
O5	потвърдено	7,765 us за пълен обмен, тоест 128 776 обмена в секунда на едно ядро

И петте очаквания се потвърждават. Това не е основание за задоволство: очакване, което се потвърждава, е било или добре обосновано, или недостатъчно смело. Полезното от измерването не е потвърждението на O1 до O5, а двете неща, които не бяха предвидени.

Първото е, че намалението на пропускателната способност при бисквитката се дължи изцяло на допълнителния обмен, а не на изчислението, и това се вижда от почти непроменения брой решения за секунда. Мерките за защита, които се обсъждат като „скъпи“, се оказват скъпи в друга единица, отколкото се предполага: в брой обмени, не в процесорно време.

Второто е, че политиката на плавно влошаване не се задейства при тази конфигурация на опита, защото бисквитката вече е свалила запълването на таблицата толкова, че режимът остава нормален. Двата механизма са проектирани независимо, но при съвместна работа единият изважда условието, при което другият има смисъл. Това е резултат, който се вижда само когато всички механизми съществуват в една система и могат да бъдат включвани поотделно, тоест точно заради решението от Раздел 2.4.

ЗАКЛЮЧЕНИЕ

Проектът разработва система за анализ на транспортната сигурност и за управление на допускането на връзки, изградена от отделни слоеве и проверена в изолирана собствена среда. Анализаторът разчита наблюдаемата част от ръкостискането по TLS 1.3 със скорост 128 776 обмена в секунда на едно ядро и отчита изрично докъде стига видимостта на пасивния наблюдател. Проверката на сертификатните вериги е реализирана самостоятелно по алгоритъма от [1], с изграждане на път и с единадесет проверки, и дава очаквания изход за всичките тринадесет построени случая. Слойта за допускане обединява четири механизма, всеки от които може да бъде включван поотделно, което позволи цената им да бъде измерена, а не предположена.

Основният измерен резултат. Собствената цена на едно решение за допускане е между 9 и 148 наносекунди, а цената на една допусната връзка под натоварване е между 454 и 1094 микросекунди. Разликата е около четири порядъка. Следователно наблюдаваното намаление на пропускателната способност при включена бисквитка, до 44,6 на сто от базовата конфигурация, не идва от изчислението, а от допълнителния обмен, който механизмът изисква. Срещу тази цена върховото запълване на таблицата с връзките пада шест пъти, а заделената памет за състояние пет пъти, при едно и също количество лавинен трафик. Мерките за защита, които се обсъждат като „скъпи“, се оказаха скъпи в брой обмени, а не в процесорно време. Това е изводът, който измерването даде и който предположението не би дало.

Предимства на решението. Разделянето на слоевете прави всеки от тях проверим самостоятелно: 82 случая покриват границите при четене, слепването през записи, строгостта на DER, всяка проверка по веригата поотделно и всеки механизъм за допускане. Работата върху генериран стенд вместо върху жив трафик прави измерванията възпроизводими и позволява да се въведе по един нарочен дефект на случай. Отсъствието на задължителни външни зависимости означава, че резултатите могат да бъдат повторени на чужда машина с две команди. И накрая, отчетът от проверката различава три състояния вместо две: проведена и успешна, проведена и неуспешна, и непроведена. Третото състояние е това, което позволява на системата да бъде честна за собствения си обхват.

Недостатъци и ограничения. Първото и най-съществено е, че проверката на подписа не се извършва. Тя изисква криптографска основа, каквато проектът съзнателно

няма, вж. Раздел 2.3. Системата не открива верига, чиито структурни полета са правилни, но подписът е подправен, и този факт е отпечатан при всяка проверка. Второто е, че пасивният анализ на TLS 1.3 вижда само част от обмена, което е свойство на протокола, а не на реализацията. Третото е, че опитите се провеждат на една машина и върху локалния интерфейс, което ограничава заключенията за поведението при действително разпределен източник: върху локалния интерфейс всички клиенти делят един адрес, така че ограничаването по източник не може да бъде оценено срещу разпределен източник. Четвъртото е, че политиката на плавно влошаване не се задейства при избраната конфигурация на опита, защото бисквитката вече е свалила запълването на таблицата под прага; поведението ѝ е проверено от тестовете, но не е измерено под натоварване. Петото е, че машината е обща, а не отделена за измерване: един ред в (Табл. 13) надхвърли своя праг за разсейване и е отбелязан като недостатъчно устойчив, вместо да бъде загладен.

Посоки за по-нататъшна работа. Първата е проверката на подписа зад условна зависимост, така че тя да се включва, когато криптографска библиотека е налична, и да продължава да се отчита като непроведена, когато не е. Втората е разчитане на формат за записан трафик, което е отделна задача за разчитане и не засяга слоя за анализ, тъй като той вече приема просто изглед към байтове. Третата е измерване в среда с действително различни източници, вместо на един адрес, което е единственият начин ограничаването по източник и политиката на плавно влошаване да бъдат оценени при условията, за които са предназначени. Четвъртата е разширяване на анализатора към QUIC, където ключовата обмяна на TLS 1.3 се пренася в друг транспорт [11], а формата на съобщенията остава същата. Петата е свързване на решенията за допускане с политиката на защитната стена по указанията в [5], така че решението да се прилага преди пакетът да достигне приложението.

ЛІТЕРАТУРА

- [1] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet x.509 public key infrastructure certificate and certificate revocation list (crl) profile,” RFC 5280, Internet Engineering Task Force, May 2008.
- [2] International Organization for Standardization, Geneva, *ISO/IEC 27033-1:2015 Information technology – Security techniques – Network security – Part 1: Overview and concepts*, 2015.
- [3] International Telecommunication Union, Geneva, *Recommendation ITU-T X.509: Information technology – Open Systems Interconnection – The Directory: Public-key and attribute certificate frameworks*, October 2019.
- [4] K. McKay and D. Cooper, “Guidelines for the selection, configuration, and use of transport layer security (tls) implementations,” NIST Special Publication 800-52 Rev. 2, National Institute of Standards and Technology, August 2019.
- [5] K. Scarfone and P. Hoffman, “Guidelines on firewalls and firewall policy,” NIST Special Publication 800-41 Rev. 1, National Institute of Standards and Technology, September 2009.
- [6] European Union Agency for Cybersecurity (ENISA), “Enisa threat landscape 2023,” tech. rep., European Union Agency for Cybersecurity, October 2023.
- [7] OWASP Foundation, *OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks*, 2021.
- [8] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, “Dns security introduction and requirements,” RFC 4033, Internet Engineering Task Force, March 2005.
- [9] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” RFC 8446, Internet Engineering Task Force, August 2018.
- [10] W. Eddy, “Tcp syn flooding attacks and common mitigations,” RFC 4987, Internet Engineering Task Force, August 2007.
- [11] M. Thomson and S. Turner, “Using tls to secure quic,” RFC 9001, Internet Engineering Task Force, May 2021.

- [12] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, “X.509 internet public key infrastructure online certificate status protocol – ocsp,” RFC 6960, Internet Engineering Task Force, June 2013.
- [13] P. Saint-Andre and J. Hodges, “Representation and verification of domain-based application service identity within internet public key infrastructure using x.509 (pkix) certificates in the context of transport layer security (tls),” RFC 6125, Internet Engineering Task Force, March 2011.

ПРИЛОЖЕНИЯ

Приложение А. Изходен текст на избрани модули

Листингите се вмъкват направо от файловете на проекта чрез `lstinputlisting`, за да останат свързани с действителния изходен текст, вместо да бъдат преписвани. Включени са шестте места, върху които стъпват твърденията от Раздел IV.

```
100     bytes_view take_u24_prefixed() { return take(u24()); }
101
102     bytes_view rest() { return take(remaining()); }
103
104 private:
105     bool read(std::size_t n) {
106         if (failed_) return false;
107         if (buf_.size() - pos_ < n) {
108             fail("need " + std::to_string(n) + " byte(s), " +
109                 std::to_string(buf_.size() - pos_) + " left");
110             return false;
111         }
112         pos_ += n;
113         return true;
114     }
115
116     bytes_view buf_;
117     std::size_t pos_ = 0;
118     bool failed_ = false;
119     std::string error_;
120 };
```

Листинг 1: Четене с проверка на границите. Отказът е лепкав, а причината за първия отказ не се презаписва от следващите (`include/sentinel/bytes.hpp`).

```
80 result<scan> scan_stream(bytes_view buf) {
81     scan s;
82     s.total_bytes = buf.size();
83     byte_reader r{buf};
84     while (r.remaining() > 0) {
```

```

85     auto rec = parse_record(r);
86     if (!rec) return result<scan>::failure(rec.error());
87     ++s.records;
88     switch (rec->type) {
89         case content_type::handshake:
90             ++s.handshake_records;
91             // Handshake messages may be split across records and several
↪ may
92             // share one, so the fragments are concatenated first and
↪ only then
93             // cut into messages.
94             s.handshake_bytes.insert(s.handshake_bytes.end(), rec->
↪ fragment.begin(),
95                                     rec->fragment.end());
96             break;
97         case content_type::application_data:
98             // Everything after the server changes keys is protected. A
↪ passive
99             // observer counts it and says so, rather than guessing at it
↪ .
100             ++s.encrypted_records;
101             s.encrypted_bytes += rec->fragment.size();
102             break;
103         case content_type::change_cipher_spec:
104             ++s.change_cipher_spec_records;
105             break;
106         case content_type::alert:
107             ++s.alert_records;
108             break;
109     }
110 }
111 if (!r.ok()) return result<scan>::failure(r.error());
112
113 byte_reader hr{bytes_view(s.handshake_bytes.data(), s.handshake_bytes.
↪ size())};
114 while (hr.remaining() > 0) {
115     message m;
116     m.type = static_cast<handshake_type>(hr.u8());
117     m.body = hr.take_u24_prefixed();

```

```

118         if (!hr.ok()) return result<scan>::failure("handshake message: " + hr
↪ .error());
119         s.messages.push_back(m);
120     }
121     return s;
122 }

```

Листинг 2: Обхождане на записите и слепване на съобщенията на ръкостискането. Едно съобщение може да се разпростре върху няколко записа, затова съдържанието се събира първо и се нарязва после (src/tls.cpp).

```

16 std::optional<tlv> reader::next() {
17     if (failed_) return std::nullopt;
18     if (pos_ >= buf_.size()) {
19         fail("no element left");
20         return std::nullopt;
21     }
22     const std::size_t start = pos_;
23     const std::uint8_t id = buf_[pos_++];
24     tlv t;
25     t.cls = static_cast<tag_class>(id >> 6);
26     t.constructed = (id & 0x20) != 0;
27     t.number = id & 0x1f;
28     if (t.number == 0x1f) {
29         // Nothing in a conforming certificate uses the high tag number form,
↪ and
30         // accepting it only widens the surface for no benefit.
31         fail("high tag number form is not accepted");
32         return std::nullopt;
33     }
34
35     if (pos_ >= buf_.size()) {
36         fail("truncated before the length byte");
37         return std::nullopt;
38     }
39     const std::uint8_t first = buf_[pos_++];
40     std::size_t len = 0;
41     if (first < 0x80) {

```

```

42     len = first;
43 } else if (first == 0x80) {
44     fail("indefinite length is not valid DER");
45     return std::nullopt;
46 } else if (first == 0xff) {
47     fail("reserved length byte 0xff");
48     return std::nullopt;
49 } else {
50     const std::size_t n = first & 0x7f;
51     if (n > sizeof(std::size_t)) {
52         fail("length field of " + std::to_string(n) + " bytes is out of
↪ range");
53         return std::nullopt;
54     }
55     if (buf_.size() - pos_ < n) {
56         fail("truncated inside a multi-byte length");
57         return std::nullopt;
58     }
59     if (buf_[pos_] == 0) {
60         fail("non-minimal length encoding, leading zero byte");
61         return std::nullopt;
62     }
63     for (std::size_t i = 0; i < n; ++i) len = (len << 8) | buf_[pos_ + i
↪ ];
64     pos_ += n;
65     if (len < 0x80) {
66         fail("non-minimal length encoding, long form used for a short
↪ length");
67         return std::nullopt;
68     }
69 }
70
71 if (buf_.size() - pos_ < len) {
72     fail("element claims " + std::to_string(len) + " byte(s), " +
73         std::to_string(buf_.size() - pos_) + " left");
74     return std::nullopt;
75 }
76 t.content = buf_.subspan(pos_, len);
77 pos_ += len;

```

```

78     t.whole = buf_.subspan(start, pos_ - start);
79     return t;
80 }

```

Листинг 3: Разчитане на един елемент по DER. Отхвърлят се неопределената дължина, неминималното кодиране и формата с висок номер на етикета (src/der.cpp).

```

276 }
277
278 void evaluator::check_signatures(const std::vector<cert_ptr>& path) {
279     // Stated once, plainly. The default build of this project has no third
    ↪ party
280     // dependency at all, and the arithmetic behind a signature check belongs
    ↪ in a
281     // reviewed cryptographic library rather than in a course project.
282     skip("signature verification",
283         "not performed: no cryptographic backend is compiled in, so the " +
284         std::to_string(path.size() - 1) +
285         " issuer signature(s) on this path were not checked");
286 }
287
288 void evaluator::run(const std::vector<cert_ptr>& path) {
289     checks.clear();
290     failed = false;
291     check_names(path);
292     check_validity(path);
293     check_basic_constraints(path);
294     check_key_usage(path);
295     check_extended_key_usage(path);
296     check_host_name(path);
297     check_name_constraints(path);
298     check_critical_extensions(path);
299     check_revocation(path);
300     check_signatures(path);
301 }

```

Листинг 4: Проверката на подписа се отчита като непроведена, с причината, редом с проверките, които са проведени (src/chain.cpp).

```

96 }
97
98 std::uint32_t make_cookie(const cookie_key& key, source_id src, std::uint64_t
↪ client_nonce,
99
100         std::uint32_t slot) {
101     // Two rounds so that neither the source nor the client value can be
↪ varied
102     // independently of the other without changing the result.
103     const std::uint64_t mixed = siphash24_u64(src ^ (static_cast<std::
↪ uint64_t>(slot) << 32),
104
105         key.k0, key.k1);
106     const std::uint64_t full = siphash24_u64(client_nonce ^ mixed, key.k0,
↪ key.k1);
107     // Truncated to 32 bits, the width a sequence number field offers. A
↪ guess
108     // succeeds with probability 2-32 per try, which is the same bound the
109     // mechanism has in the transport it is modelled on.
110     return static_cast<std::uint32_t>(full ^ (full >> 32));
111 }
112
113 bool verify_cookie(const cookie_key& key, source_id src, std::uint64_t
↪ client_nonce,
114
115         std::uint32_t cookie, std::uint32_t current_slot, std::
↪ uint32_t tolerance) {
116     // The slot has to be allowed to lag, otherwise every client whose reply
117     // crosses a slot boundary is refused.
118     for (std::uint32_t back = 0; back <= tolerance; ++back) {
119         if (back > current_slot) break;
120         if (make_cookie(key, src, client_nonce, current_slot - back) ==
↪ cookie) return true;
121     }
122     return false;
123 }

```

Листинг 5: Бисквитка без състояние. Сървърът не пази нищо между двата обмена: той пресмята стойността отново, когато клиентът я върне (src/admission.cpp).

```

155 }

```

```

156
157 std::uint64_t conn_table::open(source_id src, std::int64_t at_ns) {
158     if (entries_.size() >= capacity_) {
159         // Full. Drop the oldest connection that has not completed: an
160         ↪ incomplete
161         // connection is the cheaper thing to lose, and under a flood it is
162         // overwhelmingly likely to be part of the flood.
163         std::uint64_t oldest = 0;
164         std::int64_t oldest_ns = 0;
165         for (const auto& [id, e] : entries_) {
166             if (e.established) continue;
167             if (oldest == 0 || e.opened_ns < oldest_ns) {
168                 oldest = id;
169                 oldest_ns = e.opened_ns;
170             }
171         }
172         if (oldest == 0) return 0; // everything is established, refuse
173         ↪ instead
174         entries_.erase(oldest);
175         ++evictions_;
176     }
177     const std::uint64_t id = next_id++;
178     entries_.emplace(id, entry{src, at_ns, false});
179     peak_ = std::max(peak_, entries_.size());
180     return id;
181 }

```

Листинг 6: Изхвърляне от таблицата с връзките. При запълване си отива най-старата незавършена връзка (src/admission.cpp).

Приложение Б. Формати на структурите от данни

Изглед към байтове. `sentinel::bytes_view` е псевдоним на `std::span<const std::uint8_t>`. Нито една разчетена структура не притежава байтовете, които описва: всяко поле е изглед към буфера на извикващия. Това премахва копирането по пътя на анализа и поставя ясно изискване към времето на живот на буфера, което се проверява от тестовите.

Резултат от разчитане. `sentinel::result<T>` съдържа или стойност, или причина за отказа като низ. `std::expected` е от C++23, а проектът е на C++20.

Ръкостискане. `tls::scan` владее слепения поток от съобщения на ръкостискането и съдържа изгледи към него, заедно с броячите по вид на записа. `tls::client_hello` и `tls::server_hello` съдържат изгледи към същия буфер. `tls::handshake_report` е единственият тип тук, който владее низове, защото той е за четене от човек.

Сертификат. `x509::certificate` съдържа разчетените полета и два изгледа към изходния DER: `tbs` с точните байтове, върху които би бил изчислен подпис, и `signature` със стойността на подписа. Вторият не се чете от никого, вж. Раздел 3.2.1.

Отчет за проверката. `chain::report` съдържа присъдата, пътя и списък от `chain::check`, всеки с име, състояние от три възможни (PASS, FAIL, SKIP) и обяснение. Трите състояния са съществено в този формат: те позволяват на отчета да каже кое не е било проверено.

Запис за отчитане на връзка. Един запис в `admission::conn_table` съдържа източника, времето на отваряне и признак дали връзката е установена, като заедно с ключа заема 32 байта. Точната стойност се връща от `conn_table::bytes_per_entry()` и е величината, която се докладва в колоната за памет в (Табл. 14).

Съобщения на стенда. Заявката е 21 байта и съдържа стойността на клиента (8), бисквитката (4), решението на изчислителната задача (8) и признак за втори опит (1). Отговорът е 14 байта и съдържа присъдата (1), бисквитката (4), началната стойност на задачата (8) и трудността (1). И двете се записват побайтово с най-значимия байт напред, за да не зависят от подредбата на байтовете на машината.

Приложение В. Възпроизвеждане на опитите

Пълната последователност, изпълнена на машината от (Табл. 7):

```
git clone https://github.com/Alex-Tsvetanov/sentinel.git sentinel-network-
↳ security
cd sentinel-network-security
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
cmake --build build --target demo
./build/sentinel_bench --duration-ms 600 --repeat 11
```

Листинг 7: Сборка, проверка и измерване

За сборката са необходими единствено компилатор за C++20 и CMake от версия 3.20 нагоре. Няма управител на пакети, няма изтегляне на код при настройване и няма задължителна външна библиотека. Опитите не изискват повишени права, защото слушателят се свързва с локалния интерфейс на непривилегирован порт, избран от операционната система.

Приложение Г. Пълни резултати от измерванията

Необобщените стойности от всички единадесет повторения, върху които стъпват таблиците от Раздел VI, са записани в хранилището като docs/measurements/bench_run.txt и се възпроизвеждат по-долу без съкращения. Приложението съществува, за да може всяка обобщена стойност в изложението да бъде проследена до данните, от които е получена, включително редът, който по правилото от Раздел 5.4 е отбелязан като недостатъчно устойчив.

Следва пълният изход на командата `sentinel_bench --duration-ms 600 --repeat 11`. Изходът не носи надпис на листинг, защото се разстила върху няколко страници, а надпис в края на такъв листинг се откъсва от съдържанието си.

```
Sentinel measurement run. Defensive only: the load is generated by this
process against its own listener on the loopback interface.
```

```
machine:
```

```
compiler      : gcc 15.2.0
platform      : Windows
hardware threads: 12
build type    : optimised, assertions off
clock         : std::chrono::steady_clock, 1000000000 ticks per second
```

```
parameters:
```

```
load duration per repetition : 600 ms
repetitions per configuration: 11
honest client threads        : 2
flood client threads         : 4
```

```
=====
Parsing and validation
=====
```

operation	ns/op	ops/s	iqr	range
TLS 1.3 <code>handshake</code> scan and report	7765.4	128776	11.5%	31.9%
		300.6 MiB/s over 2448 bytes		
X.509 certificate decode	8575.7	116608	8.0%	15.8%
		109.4 MiB/s over 984 bytes		
certificate path build and validate, three certificates	17330.4	57702	6.7%	13.7%

```
individual repetitions, nanoseconds per operation:
```

```
TLS 1.3 handshake sc  8520.1 8829.2 7674.8 7527.4 7104.8 8309.7 7312.6 7522.7 7953.1 7765.4 9580.7 | median 7765.4, iqr 11.5%, range
↳ 31.9%
```

X.509 certificate de 9340.5 8575.7 8813.8 8254.7 8288.6 9275.0 8025.2 8545.7 8884.7 7982.4 9023.8 | median 8575.7, iqr 8.0%, range
 ↳ 15.8%
 certificate path bui 19022.6 17708.9 17439.5 18275.2 18294.6 16828.4 16652.1 16837.8 17271.6 17330.4 16819.4 | median 17330.4, iqr
 ↳ 6.7%, range 13.7%

=====

Admission decision cost inside the process

=====

configuration	ns/decision	iqr	range	client proof
baseline, no mechanism	8.7	40.7%	50.7%	none
accounting	93.9	8.8%	15.1%	none
accounting + rate limit	94.9	9.4%	18.7%	none
accounting + cookie	124.9	13.7%	31.9%	none
accounting + cookie + proof of work	147.8	7.3%	49.0%	0.05 ms
			4279 hashes at 12 bits	
accounting + rate limit + cookie + proof of work + degradation	136.0	32.2%	54.2%	0.06 ms
			4279 hashes at 12 bits	

individual repetitions, nanoseconds per decision:

baseline, no mechani	6.3 6.3 6.1 9.2 10.5 10.4 10.4 6.7 6.7 8.7 9.6	median 8.7, iqr 40.7%, range 50.7%
accounting	95.8 94.7 93.9 83.5 82.3 96.4 89.5 94.8 93.4 94.2 82.2	median 93.9, iqr 8.8%, range 15.1%
accounting + rate li	98.8 97.5 100.5 96.0 93.5 88.4 87.3 105.0 94.9 89.9 88.7	median 94.9, iqr 9.4%, range 18.7%
accounting + cookie	112.4 118.0 105.5 109.5 117.3 132.9 131.3 132.6 124.9 128.9 145.4	median 124.9, iqr 13.7%, range 31.9%
accounting + cookie	127.2 147.8 137.3 147.3 132.1 199.6 155.4 149.0 144.4 149.8 153.4	median 147.8, iqr 7.3%, range 49.0%
accounting + rate li	190.3 194.8 189.6 148.2 137.9 132.0 127.3 121.1 136.0 121.8 122.9	median 136.0, iqr 32.2%, range 54.2%

=====

Under flood, over loopback TCP

=====

configuration	admitted/s	p50 us	p95 us	decisions/s	peak tbl	state B
baseline, no mechanism	4305	454.2	575.9	12919	0	0
accounting	3614	524.9	762.8	10919	287	7552
accounting + rate limit	3450	516.3	858.3	10354	255	7912
accounting + cookie	1919	987.8	1390.2	11532	48	1504
accounting + cookie + proof of work	1671	1094.2	1840.1	10445	44	1408
accounting + rate limit + cookie + proof of work + degradation	1791	1027.1	1692.8	11125	43	1384

relative to the baseline configuration:

baseline, no mechanism	100.0% of baseline throughput
accounting	83.9% of baseline throughput
accounting + rate limit	80.1% of baseline throughput
accounting + cookie	44.6% of baseline throughput
accounting + cookie + proof of work	38.8% of baseline throughput
accounting + rate limit + cookie + proof of work + degradation	41.6% of baseline throughput

individual repetitions:

baseline, no mechanism	
admitted per second	4117 4163 4227 4238 4038 4469 4347 4597 4399 4383 4305 median 4305, iqr 4.6%, range 13.0%
p50 latency us	465.2 469.2 452.3 464.8 485.4 442.6 454.2 425.7 447.6 453.2 458.7 median 454.2, iqr 3.3%, range 13.1%
p95 latency us	584.3 638.4 559.0 603.2 669.5 575.9 572.5 573.2 594.9 571.3 574.1 median 575.9, iqr 4.6%, range 19.2%
decisions per second	12362 12473 12676 12693 12099 13397 13050 13774 13176 13156 12919 median 12919, iqr 4.6%, range 13.0%
totals of last run	offered 7763, admitted 2587, refused 0, peak table 0
accounting	
admitted per second	3742 3871 3448 3614 3893 3878 3790 3331 2767 2981 3376 median 3614, iqr 13.2%, range 31.1%
p50 latency us	511.2 495.0 533.1 524.9 497.5 509.0 504.9 546.4 634.7 615.2 529.8 median 524.9, iqr 6.2%, range 26.6%
p95 latency us	762.8 750.1 899.8 756.8 684.5 624.5 695.6 844.9 1077.2 1030.9 867.0 median 762.8, iqr 21.0%, range 59.3%
decisions per second	11239 11757 10363 10919 11558 11584 11408 10091 8296 8920 10066 median 10919, iqr 12.9%, range 31.7%
totals of last run	offered 6223, admitted 2087, refused 0, peak table 287
accounting + rate limit	
admitted per second	3276 3666 3801 3368 3231 3872 3743 3421 3705 3450 3028 median 3450, iqr 11.6%, range 24.5%
p50 latency us	550.8 513.0 508.4 561.8 580.0 495.2 501.4 516.3 500.4 553.0 601.0 median 516.3, iqr 10.2%, range 20.5%
p95 latency us	953.3 783.2 676.2 911.8 930.4 677.2 778.0 879.7 729.4 858.3 1026.8 median 858.3, iqr 19.5%, range 40.8%

```

    decisions per second  9766 11029 11525 10110 9658 11543 11101 10302 11303 10354 9029   | median 10354, iqr 12.2%, range 24.3%
    totals of last run    offered 5439, admitted 1824, refused 0, peak table 255
accounting + cookie
    admitted per second  1895 1974 1919 1863 1486 1423 1992 1960 1956 1928 1443   | median 1919, iqr 14.8%, range 29.6%
    p50 latency us        977.5 959.9 1000.3 1012.2 1266.0 1276.9 961.8 963.1 977.5 987.8 1258.7   | median 987.8, iqr 16.7%, range 32.1%
    p95 latency us        1286.2 1292.4 1365.1 1496.0 1977.6 2161.6 1304.8 1452.1 1379.7 1390.2 2105.0   | median 1390.2, iqr 28.9%, range
↳ 63.0%
    decisions per second  11204 11763 11532 11420 8908 8489 11879 11626 11793 11726 8659   | median 11532, iqr 14.6%, range 29.4%
    totals of last run    offered 5215, admitted 869, refused 0, peak table 48
accounting + cookie + proof of work
    admitted per second  1485 1818 1649 1738 1671 1356 1405 1722 1748 1677 1629   | median 1671, iqr 10.3%, range 27.6%
    p50 latency us        1270.2 1023.1 1091.4 1058.6 1099.5 1292.9 1303.8 1012.8 1042.7 1094.2 1101.4   | median 1094.2, iqr 12.4%, range
↳ 26.6%
    p95 latency us        2064.4 1536.8 1850.9 1627.5 1756.8 2294.4 2125.8 1536.9 1588.5 1843.0 1840.1   | median 1840.1, iqr 19.0%, range
↳ 41.2%
    decisions per second  9060 11137 10110 10885 10793 8333 8591 10693 10667 10445 10432   | median 10445, iqr 11.1%, range 26.8%
    totals of last run    offered 6396, admitted 999, refused 0, peak table 44
accounting + rate limit + cookie + proof of work + degradation
    admitted per second  1431 1522 1863 1804 1818 1777 1432 1750 1869 1791 1869   | median 1791, iqr 11.4%, range 24.5%
    p50 latency us        1296.9 1230.7 1001.6 1030.7 1027.1 1050.0 1269.8 1016.7 1002.4 1021.2 991.2   | median 1027.1, iqr 12.7%, range
↳ 29.8%
    p95 latency us        2083.8 2012.2 1544.6 1633.3 1611.1 1568.4 2110.4 1917.9 1692.8 1738.6 1664.3   | median 1692.8, iqr 20.3%, range
↳ 33.4%
    decisions per second  8790 9315 11424 11125 11281 11337 8806 10875 11506 11020 11533   | median 11125, iqr 11.6%, range 24.7%
    totals of last run    offered 6937, admitted 1124, refused 0, peak table 43

=====
Validity of this run
=====

TLS 1.3 handshake scan and report          iqr 11.5% range 31.9% within the limit
X.509 certificate decode                   iqr  8.0% range 15.8% within the limit
certificate path build and validate, three certificates iqr  6.7% range 13.7% within the limit
baseline, no mechanism                     iqr 40.7% range 50.7% TOO WIDE, do not report this row
accounting                                 iqr  8.8% range 15.1% within the limit
accounting + rate limit                     iqr  9.4% range 18.7% within the limit
accounting + cookie                         iqr 13.7% range 31.9% within the limit
accounting + cookie + proof of work          iqr  7.3% range 49.0% within the limit
accounting + rate limit + cookie + proof of work + degradation iqr 32.2% range 54.2% within the limit
baseline, no mechanism                     iqr  4.6% range 13.0% within the limit
accounting                                 iqr 13.2% range 31.1% within the limit
accounting + rate limit                     iqr 11.6% range 24.5% within the limit
accounting + cookie                         iqr 14.8% range 29.6% within the limit
accounting + cookie + proof of work          iqr 10.3% range 27.6% within the limit
accounting + rate limit + cookie + proof of work + degradation iqr 11.4% range 24.5% within the limit

at least one row exceeded its spread limit.

```